SOFTWARE TEST PLAN

HOSPITAL MANAGEMENT SYSTEM (HMS)

Document Identifier: STP-HMS-2026-V1.0

Document Version: 1.0 (Final Baseline)

Date: 20 November 2026

Course: Software Engineering Mini-Project

Prepared By (Project Team):

- **Hrushikesh Karnam** — PES1UG24CS668
- **Muruli H** — PES1UG24CS676
- **Mandagandla Jhansi** — PES1UG24CS672
- **Neeta R** — PES1UG24CS680

Department of Computer Science and Engineering

Abstract

The Hospital Management System (HMS) is a standalone, single-user C++ console application designed for hospital staff to manage patient records, doctor details, and appointment schedules using local CSV file storage. This Software Test Plan (STP) defines the testing framework, verification strategy, test environment, and test cases required to validate the system against the *HMS Software Requirements Specification (SRS) v1.0*. The plan covers functional requirements, non-functional quality attributes, and local security validation requirements. Testing focuses on validating core features, preventing doctor double-booking conflicts, handling missing or malformed data files safely, sanitizing console inputs, and preserving patient privacy. The objective of this document is to establish a systematic quality assurance process ensuring the software is reliable, secure, and ready for deployment.

Keywords: Hospital Management System, Software Testing, Test Plan, Functional Testing, Security Testing, CSV, C++

1. INTRODUCTION

1.1 Purpose

This document specifies the Software Test Plan (STP) for the Hospital Management System (HMS) v1.0. The purpose of this plan is to outline the testing scope, strategy, test levels, verification criteria, schedule, resources, and test cases necessary to verify that HMS fulfills all functional, non-functional, and security requirements defined in the *HMS Software Requirements Specification (SRS) v1.0*.

1.2 Scope

- **In Scope:** Functional testing of Patient Management (`HMS-F-001` to `HMS-F-007`), Doctor Management (`HMS-F-008` to `HMS-F-014`), Appointment Management and double-booking conflict validation (`HMS-F-015` to `HMS-F-024`), Persistence and CSV file handling (`HMS-F-025` to `HMS-F-031`), Console User Interface navigation (`HMS-F-032` to `HMS-F-036`), Non-Functional Quality Attributes (`HMS-NF-001` to `HMS-NF-010`), and Security Validation (`HMS-SR-001` to `HMS-SR-008`).
- **Out of Scope:** Billing, pharmacy, laboratory charts, web or mobile user interfaces, relational database servers, multi-user concurrency, network communications, and optional features (`HMS-O-001` to `HMS-O-004`).

1.3 References

1. *HMS Software Requirements Specification (SRS) v1.0*, Software Engineering Mini-Project, September 2026.
2. *IEEE Std 829-2008 / IEEE Std 830-1998*, IEEE Standard for Software Test Documentation.
3. *Software Engineering Mini-Project Deliverables Part-1 Instructions*, September 2026.
4. *Software Test Plan (STP) Template*, SE Mini-Project Course Materials.

1.4 Definitions and Acronyms

- **HMS:** Hospital Management System
- **SRS / STP:** Software Requirements Specification / Software Test Plan
- **RTM:** Requirements Traceability Matrix
- **FR / NFR / SR:** Functional / Non-Functional / Security Requirement
- **VR / BR:** Validation Rule / Business Rule
- **TC / CSV:** Test Case / Comma-Separated Values text file
- **UAT:** User Acceptance Testing

---

2. TEST ITEMS

The software items subject to testing are:

1. **`hms.exe`:** Compiled C++ executable built using GCC 9+ / MinGW-w64 with `-std=c++17`.
2. **Console UI Module:** Menu rendering, keyboard input parsing, and text output formatting.
3. **HospitalSystem Controller:** Core business logic engine executing validation rules, record updates, and appointment conflict validation (`BR-05`).
4. **FileManager Module:** Data storage abstraction layer handling CSV parsing, loading, and saving.
5. **Persistent Data Files:** `patients.csv`, `doctors.csv`, and `appointments.csv`.

---

3. FEATURES TO BE TESTED

The features to be tested are organized by functional area and quality category below. The complete requirement-to-test-case mapping for all individual SRS requirements (`HMS-F-001` to `HMS-F-036`, `HMS-NF-001` to `HMS-NF-010`, and `HMS-SR-001` to `HMS-SR-008`) is provided in Section 13.

Table 1. Feature-Level Summary Table

Feature Area	Features to be Tested	Requirement Type
Patient Management	Patient registration, search, viewing, updating, duplicate ID validation, input field validation	Functional
Doctor Management	Doctor registration, search, viewing, updating, duplicate ID validation, input field validation	Functional
Appointment Management	Appointment booking, availability checking, double-booking prevention, cancellation, date/time validation	Functional
Data Persistence	CSV loading, saving, missing/malformed file handling, duplicate/orphan record handling, data retention	Functional
Console Interface	Menu/submenu navigation, invalid input handling, state integrity, success/error messages, clean exit	Functional

Feature Area	Features to be Tested	Requirement Type
Non-Functional Quality	Reliability, persistence, performance, usability, maintainability, portability, correctness, standalone operation, CSV interoperability	Non-Functional
Security	Input validation, injection prevention, buffer safety, CSV validation, safe file writing, audit integrity, privacy, offline operation	Security

4. FEATURES NOT TO BE TESTED

Table 2. Features Excluded from Testing Scope

Feature / Component	Reason for Exclusion
Billing & Payments	Outside project scope; HMS focuses on records and appointments.
Pharmacy & Lab Subsystems	Detailed clinical chart subsystems are excluded from the v1.0 baseline.
Database Server (SQL)	System uses local flat-file CSV storage (`std::fstream`); no SQL database engine is used.
Web & Mobile Interface	System is designed strictly as a desktop console CLI application.
Multi-User Concurrency	Designed for single-user operation by one staff member at a time (`A-01`).
Network Protocols & TLS	Application runs completely offline without network sockets (`HMS-SR-008`).
Optional SRS Features	Features `HMS-O-001` to `HMS-O-004` are excluded unless explicitly activated.

5. TEST APPROACH / STRATEGY

Test Levels

- **Unit Testing:** Focuses on individual class methods (`Patient` , `Doctor` , `Appointment` , `FileManager`) and string validation functions (`VR-01` to `VR-14`).
- **Integration Testing:** Tests interactions between `HospitalSystem` business logic and `FileManager` CSV serialization and deserialization routines.
- **System Testing:** End-to-end verification of `hms.exe` console navigation, conflict checking, error handling, and file persistence.
- **Acceptance Testing (UAT):** Execution of standard hospital operational workflows by project team members acting as Hospital Staff.

Testing Types

- **Functional Testing:** Validates patient and doctor management, appointment booking, conflict checking, and CSV file persistence.
- **Regression Testing:** Re-executes automated and manual test suites after bug fixes to ensure existing functionality remains intact.
- **Performance Testing:** Verifies startup time (≤ 3s) and query execution time (≤ 2s) with 1,000 synthetic records (`HMS-NF-004`).
- **Usability Testing:** Evaluates prompt clarity, error message readability, and menu navigation depth (≤ 3 levels; `HMS-NF-005`).
- **Security Testing:** Verifies input sanitization, injection prevention, buffer safety, and patient data privacy (`HMS-SR-001` to `HMS-SR-008`).

- **Portability Testing:** Confirms clean compilation (`g++ -std=c++17`) and identical execution behavior on Windows and Linux (`HMS-NF-007`).

Entry Criteria

1. SRS v1.0 and design documentation are baselined.
2. C++ source code compiles without errors using `g++ -std=c++17`.
3. Test environment (console terminal, GCC compiler) is fully configured.
4. Test cases and RTM are reviewed and approved.

Exit Criteria

1. **100%** of planned functional, non-functional, and security test cases executed.
2. **100%** of High-priority defects resolved and verified closed.
3. Minimum **95%** overall test case pass rate achieved.
4. All data files maintain integrity and valid CSV format after execution.

5.1 Security Validation

Security validation is specifically designed for a local, air-gapped console application and directly links to SRS **Security Objectives (HMS-SO-01 to HMS-SO-03)** and **Security Requirements (HMS-SR-001 to HMS-SR-008)**:

1. **Accept-List Input Validation (`HMS-SR-001` / `HMS-SO-01`):** Verify all input fields are validated against domain rules (`VR-01` to `VR-14`) before altering in-memory or file state.
2. **CSV & Formula Injection Guard (`HMS-SR-002` / `HMS-SO-01`):** Test rejection of text containing commas (`,`), newlines (`\r`, `\n`), or formula prefixes (`=`, `+`, `-`, `@`).
3. **Buffer Safety (`HMS-SR-003` / `HMS-SO-02`):** Stress-test console inputs with 10,000+ character strings to ensure zero memory corruption or crashes.
4. **Untrusted CSV File Validation (`HMS-SR-004` / `HMS-SO-02`):** Test startup loading with corrupted CSV files to ensure malformed rows are skipped safely without crashing.
5. **Non-Destructive File Writes (`HMS-SR-005` / `HMS-SO-01`):** Simulate write failures (read-only CSV files) to ensure existing file content is preserved.
6. **Audit Non-Deletion (`HMS-SR-006` / `HMS-SO-01`):** Confirm cancelling an appointment retains the record in `appointments.csv` with status `Cancelled`.
7. **Patient Privacy Minimization (`HMS-SR-007` / `HMS-SO-03`):** Verify appointment list views display only Patient ID and Name, concealing address, contact number, and medical notes.
8. **Air-Gapped Isolation (`HMS-SR-008` / `HMS-SO-03`):** Verify complete offline operation with zero network socket activity.

---

6. TEST ENVIRONMENT

Table 3. Test Environment Specifications

Category	Specification / Tool	Purpose
Operating System	Windows 10 / 11 (64-bit), Ubuntu Linux 22.04 LTS	Target execution platforms (`HMS-NF-007`)
Compiler Toolchain	GCC 9+ / MinGW-w64 (`-std=c++17 -Wall`)	Build and compilation testing
IDE / Console	VS Code / Windows PowerShell / Bash	Development and test execution shell

Category	Specification / Tool	Purpose
Storage / Data	Local CSV files (`patients.csv` , `doctors.csv` , `appointments.csv`)	Persistent record storage
Test Utilities	PowerShell test harness & `Measure-Command`	Batch test execution and performance timing

7. TEST SCHEDULE

Table 4. Test Schedule Milestones

Activity	Planned Phase	Target Timeline
Test Planning & RTM Baseline	Phase 1	15-Sep-2026 – 25-Sep-2026
Environment Setup & Data Preparation	Phase 1	26-Sep-2026 – 05-Oct-2026
Unit & Integration Testing	Phase 2	06-Oct-2026 – 20-Oct-2026
System & Security Validation	Phase 2	21-Oct-2026 – 05-Nov-2026
Defect Fixing & Regression Testing	Phase 3	06-Nov-2026 – 15-Nov-2026
Final Sign-off & Deliverable Submission	Phase 3	16-Nov-2026 – 20-Nov-2026

8. TEST DELIVERABLES

1. **Software Test Plan (STP):** This IEEE-style test plan document.
2. **Requirements Traceability Matrix (RTM):** Requirement-to-Test Case mapping matrix.
3. **Test Case Suite:** Manual and script-driven test cases covering FRs, NFRs, and SRs.
4. **Test Datasets:** Baseline, edge-case, malformed, and 1,000-record synthetic CSV files.
5. **Test Execution & Defect Reports:** Recorded console execution logs and defect reports.
6. **Test Summary Report:** Executive summary presenting planned vs actual testing metrics.

9. ROLES AND RESPONSIBILITIES

Table 5. Team Roles and Responsibilities

Role	Team Member	Primary Responsibilities
QA Lead	Hrushikesh Karnam	Test plan creation, test strategy definition, RTM maintenance, security validation.
Test Engineer	Muruli H	Test case design, appointment conflict matrix testing, CLI menu validation.
Developer Lead	Mandagandla Jhansi	Unit and integration testing, CSV file handling verification, defect fixing.
Support Lead	Neeta R	Test data generation, performance timing, cross-platform build testing.

10. RISKS AND MITIGATION

Table 6. Risk Analysis and Mitigation Strategies

Risk Description	Impact	Mitigation Strategy
1. C++ Build / Logic Delay: Conflict check logic defect delays testing.	Medium	Use driver stubs to test `FileManager` and `Console UI` independently.
2. CSV File Corruption: File corruption during test crashes.	Medium	Maintain read-only backup copies of all test CSV datasets.
3. OS Line-Ending Differences: `\r\n` vs `\n` causing parse errors.	Low	Enforce `VR-14` text sanitization and explicit text-mode file streams.
4. Search Latency under Load: Latency exceeding 2 seconds for 1,000 items.	Low	Perform early in-memory lookup profiling on standard vectors.
5. Testing Time Constraints: Limited mini-project submission timeline.	High	Prioritize high-priority FRs, conflict detection, and security test cases.

11. ASSUMPTIONS AND DEPENDENCIES

11.1 Assumptions

1. Single-user execution environment operated by one Hospital Staff member at a time (`A-01`).
2. Operating system user has read/write permissions in the local application directory.
3. Availability rules depend strictly on existing non-cancelled appointments (`BR-05`).

11.2 Dependencies

1. Stability of frozen SRS v1.0 specifications.
2. Availability of standard GCC C++17 compiler toolchain.

12. SUSPENSION AND RESUMPTION CRITERIA

12.1 Suspension Criteria

Testing activities shall be suspended if:

1. The C++ build fails to compile or crashes immediately upon startup.
2. A critical blocking defect prevents execution of > 30% of remaining test cases.
3. Test environment or dataset files become unrecoverably corrupted.

12.2 Resumption Criteria

Testing activities shall resume when:

1. A corrected build is provided and passes smoke testing.
2. The blocking defect is verified fixed by the QA Lead.
3. Clean test CSV datasets are restored.

13. TEST CASE MANAGEMENT AND TRACEABILITY

Requirements Traceability Matrix (RTM)

Table 7. Requirements Traceability Matrix

Requirement ID	Requirement Summary	Test Case ID	Test Type	Expected Result
HMS-F-001	Register Patient	`TC-PAT-01`	Functional	Patient record created & saved in `patients.csv`.
HMS-F-002	Reject Duplicate Patient ID	`TC-PAT-02`	Functional	Error message displayed; duplicate ID rejected.
HMS-F-003	Validate Patient Fields (`VR-01` – `VR-07`)	`TC-PAT-03`	Functional / Boundary	Out-of-range inputs rejected; valid data accepted.
HMS-F-004	Search Patient (ID / Name)	`TC-PAT-04`	Functional	Exact ID and partial name return matching records.
HMS-F-005	View Patient Details & List	`TC-PAT-05`	Functional	Displays all stored fields for selected patient.
HMS-F-006	Update Patient Details	`TC-PAT-06`	Functional	Updated values saved to memory and CSV file.
HMS-F-007	Prevent Patient ID Edit	`TC-PAT-07`	Functional	Patient ID field remains uneditable during update.
HMS-F-008	Add Doctor	`TC-DOC-01`	Functional	Doctor record created & saved in `doctors.csv`.
HMS-F-009	Reject Duplicate Doctor ID	`TC-DOC-02`	Functional	Error message displayed; duplicate ID rejected.
HMS-F-010	Validate Doctor Fields (`VR-08` – `VR-09`)	`TC-DOC-03`	Functional / Boundary	Invalid specialization/contact rejected.
HMS-F-011	Search Doctor (ID/Name/Spec)	`TC-DOC-04`	Functional	Returns exact doctor matches across search modes.
HMS-F-012	View Doctor Details & List	`TC-DOC-05`	Functional	Displays complete doctor record details.
HMS-F-013	Update Doctor Details	`TC-DOC-06`	Functional	Doctor details updated successfully.
HMS-F-014	Prevent Doctor ID Edit	`TC-DOC-07`	Functional	Doctor ID field remains uneditable during update.
HMS-F-015	Book Appointment	`TC-APT-01`	Functional	Booking succeeds with system-generated ID (`A0001`).
HMS-F-016	Reject Invalid Patient FK	`TC-APT-02`	Functional	Rejects booking for non-existent Patient ID.
HMS-F-017	Reject Invalid Doctor FK	`TC-APT-03`	Functional	Rejects booking for non-existent Doctor ID.
HMS-F-018	Validate Date & Time Format	`TC-APT-04`	Functional / Boundary	Invalid date (`2026-02-30`) & time (`24:00`) rejected.

Requirement ID	Requirement Summary	Test Case ID	Test Type	Expected Result
HMS-F-019	**Doctor Double-Booking Conflict**	`TC-APT-05`	Functional / Logic	**Rejects second booking for same doctor at same slot.**
HMS-F-020	Check Doctor Availability	`TC-APT-06`	Functional	Reports "Available" or "Not Available" accurately.
HMS-F-021	Exclude Cancelled from Conflict	`TC-APT-05`	Functional / Logic	Re-booking slot succeeds after cancellation.
HMS-F-022	Sorted Appointment Views	`TC-APT-08`	Functional	Appointments listed in chronological date/time order.
HMS-F-023	Cancel Appointment	`TC-APT-09`	Functional	Status updated to `Cancelled`; record retained in CSV.
HMS-F-024	Reject Invalid Cancellation	`TC-APT-10`	Functional	Cancelling non-existent or cancelled ID rejected.
HMS-F-025	Startup CSV Data Loading	`TC-PER-01`	Functional / Integration	Pre-existing CSV records loaded into memory on launch.
HMS-F-026	Missing CSV File Graceful Load	`TC-PER-02`	Functional / Robustness	Missing CSV treated as empty; created on save.
HMS-F-027	Skip Malformed CSV Lines	`TC-PER-03`	Functional / Robustness	Skips corrupted lines and reports count without crash.
HMS-F-028	Skip Duplicate/Orphan Records	`TC-PER-04`	Functional / Integration	Duplicate IDs and orphan appointments ignored on load.
HMS-F-029	Immediate CSV Persistence Writing	`TC-PER-05`	Functional / Integration	CSV updated immediately after each write operation.
HMS-F-030	Restart Record Retention	`TC-PER-06`	Functional / Persistence	Data retained 100% across app close and restart.
HMS-F-031	Read-Only Write Error Handling	`TC-PER-07`	Functional / Reliability	Displays error message when CSV write fails.
HMS-F-032	Menu Navigation Hierarchy	`TC-UI-01`	Functional	Main menu and submenus navigate correctly.
HMS-F-033	Invalid Menu Option Rejection	`TC-UI-02`	Functional / Robustness	Non-numeric and out-of-range menu inputs handled.
HMS-F-034	Transactional State Protection	`TC-UI-03`	Functional / Integrity	In-memory state unchanged when operation fails.
HMS-F-035	Standard Outcome Messaging	`TC-UI-04`	Functional	Clear success confirmation or error message displayed.
HMS-F-036	Clean Application Exit	`TC-UI-05`	Functional	System exits cleanly with code 0.

Requirement ID	Requirement Summary	Test Case ID	Test Type	Expected Result
HMS-NF-001	Input Crash Resilience	`TC-NFR-01`	Reliability	System survives 20+ invalid keyboard input cases.
HMS-NF-002	Corrupted Startup Resilience	`TC-NFR-02`	Reliability	Starts successfully with missing/corrupted files.
HMS-NF-003	Data Retention Rate	`TC-NFR-03`	Persistence	100% record retention verified after restart.
HMS-NF-004	Load & Execution Latency	`TC-NFR-04`	Performance	Startup ≤ 3s, operations ≤ 2s with 1,000 records.
HMS-NF-005	UI Usability & Menu Depth	`TC-NFR-05`	Usability	Prompts state format; max menu depth ≤ 3.
HMS-NF-006	Architectural Class Isolation	`TC-NFR-06`	Maintainability	5 OOP classes verified; file I/O isolated to `FileManager`.
HMS-NF-007	Cross-Platform Compilation	`TC-NFR-07`	Portability	Compiles clean with `-std=c++17` on Windows & Linux.
HMS-NF-008	100% Conflict Matrix Detection	`TC-NFR-08`	Correctness	All 4 conflict matrix scenarios evaluate accurately.
HMS-NF-009	Standalone Deployment	`TC-NFR-09`	Deployment	Runs fully offline without network or external DB.
HMS-NF-010	CSV Interoperability	`TC-NFR-10`	Interoperability	CSV files correctly formatted with headers.
HMS-SR-001	Accept-List Input Validation	`TC-SEC-01`	Security	Rejects inputs violating `VR-01` to `VR-14`.
HMS-SR-002	CSV & Formula Injection Guard	`TC-SEC-02`	Security	Rejects inputs containing `,`, `\r`, `\n`, `=`, `+`, `-`, `@`.
HMS-SR-003	Over-Length Buffer Safety	`TC-SEC-03`	Security	System handles 10,000-char input safely.
HMS-SR-004	Untrusted CSV File Validation	`TC-SEC-04`	Security	Invalid fields in CSV skipped entirely on load.
HMS-SR-005	Non-Destructive File Writes	`TC-PER-07`	Security	Previous file content retained on write failure.
HMS-SR-006	Non-Deletion Audit Integrity	`TC-APT-09`	Security	Cancelled appointment row preserved in CSV.
HMS-SR-007	Patient Data Privacy Protection	`TC-SEC-07`	Security	Confidential fields omitted from appointment views.
HMS-SR-008	Air-Gapped Offline Operation	`TC-NFR-09`	Security	Application operates with zero network activity.

Detailed Test Cases (10 Representative Specifications)

Table 8. Test Case Specification 1 — Patient Registration

Attribute	Specification Detail
Test Case ID	`TC-PAT-01`
Requirement ID	`HMS-F-001`
Test Objective	Verify patient registration with valid data and persistent CSV output.
Test Type	Positive Functional
Preconditions	`hms.exe` is running; Patient `P001` is not present in `patients.csv`.
Test Data	`P001`, `John Doe`, `45`, `M`, `9876543210`, `123 Main St`, `Hypertension`
Test Steps	1. Navigate to **Patient Management -> Register Patient**. 2. Enter Patient ID: `P001`. 3. Enter Name: `John Doe`, Age: `45`, Gender: `M`. 4. Enter Contact: `9876543210`, Address: `123 Main St`, Medical Note: `Hypertension`.
Expected Result	Confirmation message displayed; `patients.csv` contains the new patient record row.

Table 9. Test Case Specification 2 — Duplicate Patient ID Rejection

Attribute	Specification Detail
Test Case ID	`TC-PAT-02`
Requirement ID	`HMS-F-002`
Test Objective	Verify that registering a duplicate Patient ID is rejected.
Test Type	Negative Functional
Preconditions	Patient `P001` is already registered in `patients.csv`.
Test Data	`p001`, `Jane Smith`, `30`, `F`, `9123456789`, `456 Park Ave`, `None`
Test Steps	1. Navigate to **Patient Management -> Register Patient**. 2. Enter Patient ID: `p001` (case-insensitive test). 3. Enter remaining valid fields.
Expected Result	System displays error `"Patient ID already exists"`; no changes written to `patients.csv`.

Table 10. Test Case Specification 3 — Appointment Booking

Attribute	Specification Detail
Test Case ID	`TC-APT-01`
Requirement ID	`HMS-F-015`
Test Objective	Verify appointment booking with system-generated ID `A0001`.
Test Type	Positive Functional
Preconditions	Patient `P001` and Doctor `D001` exist; `D001` has no appointments on `2026-10-15` at `10:00`.

Attribute	Specification Detail
Test Data	`P001`, `D001`, `2026-10-15`, `10:00`
Test Steps	1. Navigate to **Appointment Management -> Book Appointment**. 2. Enter Patient ID: `P001`, Doctor ID: `D001`. 3. Enter Date: `2026-10-15`, Time: `10:00`.
Expected Result	Confirmation displayed with ID `A0001`; `appointments.csv` updated with status `Booked`.

Table 11. Test Case Specification 4 — Doctor Double-Booking Conflict Rejection & Cancelled Slot Re-booking

Attribute	Specification Detail
Test Case ID	`TC-APT-05`
Requirement ID	`HMS-F-019`, `HMS-F-021`, `BR-05`, `BR-06`
Test Objective	Verify conflict rejection for double-booking and slot re-booking after cancellation.
Test Type	Logic & Conflict Matrix Test
Preconditions	Doctor `D001` has active appointment `A0001` on `2026-10-15` at `10:00`.
Test Data	Step 1: `P002`, `D001`, `2026-10-15`, `10:00`. Step 2: Cancel `A0001`. Step 3: `P002`, `D001`, `2026-10-15`, `10:00`.
Test Steps	1. Attempt second booking for Doctor `D001` at `2026-10-15 10:00`. Verify rejection. 2. Cancel appointment `A0001`. 3. Re-book Doctor `D001` at `2026-10-15 10:00` for Patient `P002`. Verify acceptance.
Expected Result	Step 1 fails with `"Doctor not available at the requested date and time"`. Step 3 succeeds with new ID `A0002` after cancellation frees the slot.

Table 12. Test Case Specification 5 — Date & Time Field Validation

Attribute	Specification Detail
Test Case ID	`TC-APT-04`
Requirement ID	`HMS-F-018`, `VR-10`, `VR-11`
Test Objective	Verify rejection of invalid appointment date and time inputs.
Test Type	Boundary Validation
Preconditions	Patient `P001` and Doctor `D001` exist in system.
Test Data	Case 1: `2026-02-30`, `10:00`. Case 2: `2026-10-15`, `24:15`.
Test Steps	1. Attempt booking with Date `2026-02-30` (non-existent calendar day). 2. Attempt booking with Time `24:15` (invalid 24-hour time).
Expected Result	Both attempts are rejected with specific format error messages (`VR-10` / `VR-11`).

Table 13. Test Case Specification 6 — Missing CSV File Startup

Attribute	Specification Detail
Test Case ID	`TC-PER-02`
Requirement ID	`HMS-F-026`, `HMS-NF-002`
Test Objective	Verify graceful startup when data files are missing from the working folder.
Test Type	Robustness Test
Preconditions	Delete `patients.csv`, `doctors.csv`, and `appointments.csv` from local folder.
Test Data	Standard registration input for Patient `P001`.
Test Steps	1. Delete all CSV files. 2. Launch `hms.exe`. Verify main menu displays. 3. Register Patient `P001` and exit application.
Expected Result	Application starts normally without crashing, treating datasets as empty. `patients.csv` is created on first save.

Table 14. Test Case Specification 7 — CSV & Formula Injection Guard

Attribute	Specification Detail	
Test Case ID	`TC-SEC-02`	
Requirement ID	`HMS-SR-002`, `HMS-SO-01`	
Test Objective	Verify rejection of input containing CSV delimiters and formula prefixes.	
Test Type	Security Validation	
Preconditions	Application running at Patient Registration prompt.	
Test Data	Input 1: `Doe, John`. Input 2: `=cmd	' /C calc'!A0`.
Test Steps	1. Attempt registration with Patient Name `Doe, John` (contains comma). 2. Attempt registration with Patient Name `=cmd	' /C calc'!A0` (formula prefix).
Expected Result	Inputs are rejected with message `"Input contains illegal character (comma, newline, or formula prefix =, +, -, @)"`.	

Table 15. Test Case Specification 8 — Patient Data Privacy Minimization

Attribute	Specification Detail
Test Case ID	`TC-SEC-07`
Requirement ID	`HMS-SR-007`, `HMS-SO-03`
Test Objective	Verify appointment screens conceal sensitive patient contact, address, and medical notes.
Test Type	Privacy Verification

Attribute	Specification Detail
Preconditions	Patient `P001` (`John Doe` , Contact: `9876543210` , Address: `123 Main St` , Note: `HIV Positive`) has appointment `A0001` .
Test Data	Option 1 (View All Appointments)
Test Steps	1. Navigate to **Appointment Management -> View Appointments -> View All Appointments**. 2. Inspect displayed fields.
Expected Result	Screen displays `Appointment ID` , `Patient ID/Name` , `Doctor ID/Name` , `Date` , `Time` , and `Status` . Confidential fields (`Contact` , `Address` , `Medical Note`) are omitted.

Table 16. Test Case Specification 9 — Performance Benchmark Under Load

Attribute	Specification Detail
Test Case ID	`TC-NFR-04`
Requirement ID	`HMS-NF-004`
Test Objective	Measure startup loading time and booking search latency with 1,000 synthetic records per file.
Test Type	Performance Benchmark
Preconditions	CSV files populated with 1,000 valid records each.
Test Data	1,000 synthetic rows per file
Test Steps	1. Measure launch time from process start to main menu display. 2. Execute appointment booking with conflict check for doctor at slot 1,000. 3. Measure response latency using `Measure-Command` .
Expected Result	Startup file loading completes in **< 3.0 seconds**; appointment booking and conflict check complete in **< 2.0 seconds**.

Table 17. Test Case Specification 10 — Console Input Crash Resilience

Attribute	Specification Detail
Test Case ID	`TC-NFR-01`
Requirement ID	`HMS-NF-001`
Test Objective	Stress-test console menus with 20 distinct invalid input vectors.
Test Type	Reliability Test
Preconditions	Application running at Main Menu prompt.
Test Data	Matrix of 20 invalid inputs (`""` , `"abc"` , `"-5"` , `"999999"` , `"%x%s"` , etc.)
Test Steps	1. Execute automated test script submitting 20 invalid input sequences. 2. Monitor process return code and console log output.
Expected Result	Zero application crashes (0 crash rate across all 20 test vectors); menus re-prompt cleanly with error messages.

14. TEST METRICS AND REPORTING

Test Metrics and Measurement Criteria

- **Execution Rate (%)** = $(\text{Executed Test Cases} / \text{Total Planned Test Cases}) \times 100$

Target: 100% of planned test cases executed.

- **Pass Rate (%)** = $(\text{Passed Test Cases} / \text{Executed Test Cases}) \times 100$

Target: $\ge 95\%$.

- **Requirement Coverage (%)** = $(\text{Requirements Mapped to Test Cases} / \text{Total Baseline Requirements}) \times 100$

Planned Coverage: 100%.

- **Defect Count:** Total number of software defects identified during test execution.
- **Defect Severity:** Categorized as High/Critical, Medium, and Low.
- **Defect Density:** Number of defects relative to defined test scope/size, where applicable.
- **Defect Aging:** Time taken to resolve open defects from logging to closure.
- **Defect Closure:** Verification and tracking of defect resolution status to ensure all high/critical issues are closed before deployment.

Planned Requirement Coverage

Table 18. Planned Requirement Coverage Summary

Category	Total Baseline Requirements	Requirements Mapped to Test Cases	Planned Coverage (%)
Functional Requirements (FR)	36 (`HMS-F-001` – `HMS-F-036`)	36	**100.0%**
Non-Functional Requirements (NFR)	10 (`HMS-NF-001` – `HMS-NF-010`)	10	**100.0%**
Security Requirements (SR)	8 (`HMS-SR-001` – `HMS-SR-008`)	8	**100.0%**
Total Baseline Requirements	**54**	**54**	**100.0%**

Test Reporting Outputs

- **Daily Test Execution Status:** Progress summary detailing daily test cases executed and open defects.
- **Defect Report:** Defect tracking log detailing steps to reproduce, expected vs actual behavior, severity, and resolution status.
- **Final Test Summary Report:** Executive report presenting final test metrics, defect closure verification, and deployment readiness recommendation.

---

15. APPROVALS

The signatures below indicate formal approval and sign-off of this Software Test Plan (STP) baseline document. The signature and date fields remain blank pending final review and evaluator sign-off.

Table 19. Document Sign-off and Approval

Role	Team Member / Evaluator	Signature / Approval	Date
QA Lead / Student	Hrushikesh Karnam		
Test Engineer / Student	Muruli H		
Developer Lead / Student	Mandagandla Jhansi		
Support Lead / Student	Neeta R		
Course Coordinator / Evaluator	Mini-Project Evaluator		